Assignment #3: Performance Analysis of RVV Vectorization and Memory Access Patterns

Žane Bučan, Stjepan Sazonov
RS 2025/26 , FRI, UL

I. Introduction

This report analyzes the performance of vectorized RISC-V Vector (RVV) implementations for three workloads: scaled dot-product attention, a one-dimensional heat-diffusion stencil, and sparse matrix-vector multiplication (SpMV). The goal is to evaluate how vector processing unit (VPU) width, L1 cache size, and memory access pattern affect performance.

II. Task 1: Vectorizing Kernel

A. Scaled Dot-Product Attention

The first workload computes scaled dot-product attention scores.

VPU (bits)	Scalar CPI	Scalar Time	Vector CPI	Vector Time
128	0.775324	0.000110	2.491596	0.000852
256	0.775324	0.000110	2.460773	0.000451
512	0.775324	0.000110	2.412990	0.000251
1024	0.775324	0.000110	2.329477	0.000150
2048	0.775324	0.000110	2.229050	0.000099
4096	0.775324	0.000110	2.227282	0.000099

TABLE I
Scaled dot-product attention performance with 8KB L1 cache

The scalar implementation has a lower CPI than the vectorized implementation for all tested VPU sizes. However, CPI is not directly comparable between scalar and vector execution, because one vector instruction operates on multiple data elements. Therefore, execution time is the more important metric for evaluating the benefit of RVV vectorization.

The optimized vectorized implementation shows a clear improvement as the VPU size increases. Execution time decreases from 0.000852 at 128 bits to 0.000099 at 2048 and 4096 bits. This indicates that wider vector units amortize the overhead of vector setup and reductions more effectively. At smaller VPU sizes, the vectorized version is slower than the scalar baseline because the workload is small and vector overhead is still significant. At 2048 bits and above, the vectorized version becomes slightly faster than the scalar version.

The speedup of the 2048-bit vectorized implementation compared to the scalar baseline is:

$$\frac{0.000110}{0.000099} \approx 1.11$$

which corresponds to an approximately 11% reduction in execution time.

III. Task 2: Stencil Kernel

A. Heat Diffusion Stencil

The second workload computes one time step of a one-dimensional heat diffusion equation using a 3-point stencil:

$$u_{new}[i] = u[i] + \alpha \cdot (u[i-1] - 2 \cdot u[i] + u[i+1]).$$

Each interior point requires three neighboring values, so the workload has a regular memory access pattern with substantial data reuse.

B. 8KB L1 Cache Results

VPU (bits)	Scalar CPI	Scalar Misses	Vector CPI	Vector Misses
128	0.847457	65	2.547448	50
256	0.847457	65	2.568728	52
512	0.847457	65	2.612624	55
1024	0.847457	65	2.676691	68
2048	0.847457	65	2.810465	61
4096	0.847457	65	3.901307	355

TABLE II
Stencil performance with 8KB L1 cache

With an 8KB L1 cache, the scalar stencil implementation remains stable at 0.847457 CPI and 65 L1 misses. The vectorized implementation has a higher CPI for all VPU sizes. Up to 2048 bits, the vectorized L1 miss count remains close to the scalar result, but at 4096 bits the miss count rises sharply to 355 and CPI increases to 3.901307.

This result indicates that very wide vectors can increase cache pressure. The 4096-bit configuration appears too large for the 8KB L1 cache in this workload, causing significantly more cache misses and reducing performance.

C. 64KB L1 Cache Results

VPU (bits)	Scalar CPI	Scalar Misses	Vector CPI	Vector Misses
128	0.827531	0	2.539969	42
256	0.827531	0	2.552192	42
512	0.827531	0	2.578073	42
1024	0.827531	2	2.608516	44
2048	0.827531	6	2.677283	48
4096	0.827531	9	2.805427	51

TABLE III
STENCIL PERFORMANCE WITH 64KB L1 CACHE

Increasing the L1 cache from 8KB to 64KB significantly improves cache behavior. The scalar implementation has almost no L1 misses, while the vectorized implementation remains between 42 and 51 misses across all VPU sizes. The largest improvement appears at 4096 bits, where vectorized CPI decreases from 3.901307 with 8KB L1 to 2.805427 with 64KB L1.

The larger cache reduces the penalty of wide vector accesses and avoids the severe degradation observed with 4096-bit vectors in the 8KB configuration.

IV. Task 3: Sparse Matrix-Vector Multiplication

A. Memory Access Patterns

The third workload evaluates sparse matrix-vector multiplication using four RVV kernels with different memory access patterns: unit-stride, strided, gather with sorted indices, and gather with random indices.

B. 8KB L1 Cache Results

Kernel	VPU	CPI	L1 Misses
Unit-stride	256	2.504482	32
Unit-stride	512	2.572486	41
Unit-stride	1024	2.711817	44
Strided	256	2.562149	107
Strided	512	3.579013	613
Strided	1024	4.276478	820
Gather sorted	256	2.453685	37
Gather sorted	512	2.558479	53
Gather sorted	1024	2.647485	69
Gather random	256	3.106025	515
Gather random	512	3.363072	535
Gather random	1024	3.866654	569

TABLE IV
SPMV PERFORMANCE WITH 8KB L1 CACHE

With an 8KB cache, unit-stride and sorted gather have the best performance. Unit-stride accesses are contiguous, while sorted gather accesses are predictable enough for the cache hierarchy to handle efficiently. Strided and random gather patterns perform worse because they reduce spatial locality and make prefetching less effective.

The strided kernel is especially sensitive to VPU size. Its L1 misses increase from 107 at 256 bits to 820 at 1024 bits, and CPI increases from 2.562149 to 4.276478. Random gather also has many misses, but its miss count grows more slowly across the tested VPU sizes.

C. 64KB L1 Cache Results

Kernel	VPU	CPI	L1 Misses
Unit-stride	256	2.464176	22
Unit-stride	512	2.542392	30
Unit-stride	1024	2.602150	31
Strided	256	2.549792	95
Strided	512	3.577364	604
Strided	1024	4.289290	821
Gather sorted	256	2.439296	28
Gather sorted	512	2.537564	44
Gather sorted	1024	2.572118	61
Gather random	256	2.431806	2
Gather random	512	2.467215	2
Gather random	1024	2.524782	4

TABLE V
SPMV PERFORMANCE WITH 64KB L1 CACHE

The 64KB L1 cache improves most patterns, but the effect is not uniform. Unit-stride and sorted gather improve slightly because they already have good locality with 8KB. The largest improvement is observed for random gather: at 256 bits, CPI decreases from 3.106025 to 2.431806 and L1 misses drop from 515 to 2. At 1024 bits, CPI decreases from 3.866654 to 2.524782 and L1 misses drop from 569 to 4.

The results show that memory access pattern is a dominant factor in SpMV performance. Contiguous and predictable accesses achieve lower CPI and fewer misses, while strided and random accesses suffer from poor locality. However, the 64KB cache is large enough to almost completely eliminate misses for the random gather case in this benchmark.

V. Conclusion

The experiments show that vectorization does not automatically guarantee faster execution. In the scaled dot-product attention workload, the vectorized version has higher CPI and higher execution time than the scalar baseline, although wider VPUs reduce the vectorized execution time up to 2048 bits.

For the stencil workload, cache size has a strong impact on wide-vector performance. With an 8KB L1 cache, the 4096-bit VPU configuration causes a sharp increase in L1 misses and CPI. With a 64KB L1 cache, this degradation is largely removed, showing that the wider vector configuration benefits from additional cache capacity.

For SpMV, performance is primarily determined by memory access regularity. Unit-stride and sorted gather patterns perform well because they preserve locality and predictability. Strided access performs poorly even with a larger cache, while random gather improves dramatically with a 64KB cache. Overall, the results demonstrate that RVV performance depends not only on vector width, but also on cache capacity, memory locality, and the ability of the workload to amortize vector overhead.